

prog_datasci_4_pandas

September 24, 2022

1 Fonaments de Programació

1.1 Unitat 4: Llibreries científiques en Python - pandas

1.1.1 Instruccions d'ús

Aquest document és un *notebook* interactiu que intercala explicacions més aviat teòriques de conceptes de programació amb fragments de codi executables. Per aprofitar els avantatges que aporta aquest format, us recomanem que, en primer lloc, llegiu les explicacions i el codi que us proporcionem. D'aquesta manera tindreu un primer contacte amb els conceptes que hi exposem. Ara bé, **la lectura és només el principi!** Una vegada hàgiu llegit el contingut, no oblideu executar el codi proporcionat i modificar-lo per crear-ne variants que us permetin comprovar que heu entès la seva funcionalitat i explorar-ne els detalls d'implementació. Per últim, us recomanem també consultar la documentació enllaçada per explorar amb més profunditat les funcionalitats dels mòduls presentats.

Per guardar possibles modificacions que feu sobre aquest notebook, us aconsellem que munteu la unitat de Drive a Google Colaboratory (colab). Heu d'executar les instruccions següents:

```
[ ]: from google.colab import drive  
drive.mount('/content/drive')
```

```
[ ]: %cd /content/drive/MyDrive/Colab_Notebooks/prog_datasci_4
```

1.1.2 Introducció

A continuació es presentaran explicacions i exemples d'ús de la llibreria pandas. Recordeu que podeu anar executant els exemples per obtenir-ne els resultats.

A continuació s'inclou la taula de continguts, que podeu fer servir per navegar pel document:

1 Introducció

2	Estructures de dades bàsiques
3	Operacions bàsiques sobre un dataframe
3.1	Lectura de dades d'un fitxer
3.2	Exploració del dataframe
3.3	Indexació i selecció de dades
3.4	Agregació de dades
3.5	Escriptura de dades a un fitxer
4	Exercicis i preguntes teòriques
4.1	Instruccions importants
# 1	Introducció

Pandas és una llibreria de Python que ens ofereix una interfície d'alt nivell per manipular i analitzar dades. La llibreria ofereix estructures de dades flexibles sobre les quals carregar-les i implementa operacions sobre aquestes estructures l'ús de les quals resulta molt intuïtiu i que, alhora, són eficients. Per aquest motiu, és una de les eines més utilitzades en l'anàlisi de dades en Python. Pandas també forma part del SciPy stack.

Com veurem més endavant, l'estructura de dades principal de Pandas és el dataframe, una estructura que permet emmagatzemar taules bidimensionals.

Per poder utilitzar la llibreria, en primer lloc cal importar-la:

```
[1]: # A la línia següent, importem pandas i li donem un nom més curt
# perquè ens sigui més còmode fer les crides.
import pandas as pd
# Importem també NumPy, ja que la farem servir en algun dels exemples.
import numpy as np
```

2 Estructures de dades bàsiques

Pandas proveeix de tres estructures de dades: la sèrie, el *dataframe* i antigament el panell (ara, els *dataframes* i les *sèries* amb índexs multinivell). Vegem les característiques de cadascuna.

Una **sèrie** és un vector **unidimensional** amb **etiquetes** als eixos i dades **homogènies**.

Repassem què impliquen cadascuna d'aquestes característiques amb uns exemples.

La sèrie ens permet representar un conjunt de dades unidimensionals, per exemple, una llista d'enters, decimals o de cadenes de caràcters:

```
[2]: print(pd.Series([1, 1, 2, 3, 5]))
```

```
0    1
1    1
2    2
3    3
```

```
4      5
dtype: int64
```

```
[3]: print(pd.Series([1.5, 3.5, 4.75]))
```

```
0      1.50
1      3.50
2      4.75
dtype: float64
```

Les dades d'una sèrie han de ser homogènies, és a dir, han de ser del mateix tipus. Als exemples anteriors, la primera sèrie estava formada per enters (int64), mentre que la segona contenia nombres en punt flotant (float).

De totes maneres, si intentem crear una sèrie amb dades de diferents tipus, podem fer-ho, ja que pandas crearà una sèrie amb el tipus més general:

```
[4]: # Barregem enters i 'floats', la sèrie és de tipus 'float'.
print(pd.Series([1, 2, 3.5]))
```

```
0      1.0
1      2.0
2      3.5
dtype: float64
```

```
[5]: # Barregem enters, 'floats' i 'strings', la sèrie és de tipus 'object'.
print(pd.Series([1, 4.3, "data"]))
```

```
0      1
1      4.3
2      data
dtype: object
```

Finalment, la sèrie pot tenir etiquetes, de manera que podem accedir als elements d'una sèrie tant a partir del seu índex com de la seva etiqueta.

```
[6]: # Creem una sèrie etiquetada a partir d'un diccionari.
s = pd.Series({"alice" : 2, "bob": 3, "eve": 5})
print(s)

# Accedim als elements de la sèrie a partir de la seva etiqueta.
print(s["alice"])

# Accedim als elements de la sèrie a partir del seu índex.
print(s[0])
```

```
alice      2
bob        3
eve        5
dtype: int64
```

2
2

```
[7]: # Creem una sèrie etiquetada a partir de dos vectors, un amb les dades i un
      ↳altre amb les etiquetes.
print(pd.Series([2, 3, 5], index = ["alice", "bob", "eve"]))
```

```
alice    2
bob      3
eve      5
dtype: int64
```

La segona estructura de dades de pandas que presentarem és el *dataframe*.

Un *dataframe* és una taula **bidimensional** amb **etiquetes** als eixos i dades potencialment **heterogènies**. El *dataframe* és l'estructura principal de treball amb la llibreria pandas.

Vegem les característiques principals d'un *dataframe* amb alguns exemples.

A diferència d'una sèrie, un *dataframe* és bidimensional:

```
[8]: print(pd.DataFrame([[1, 2, 3], [4, 5, 6]]))
```

```
   0  1  2
0  1  2  3
1  4  5  6
```

Igual que la sèrie, el *dataframe* pot tenir etiquetes als eixos i podem utilitzar diferents sintaxis per incloure les etiquetes al *dataframe*.

```
[9]: # Fem servir un diccionari per definir cada columna i una llista per indicar les
      ↳etiquetes de les files.
d = {"alice" : [1953, 12, 3], "bob" : [1955, 11, 24], "eve" : [2011, 10, 10]}
print(pd.DataFrame(d, index=["year", "month", "day"]))
```

```
      alice  bob  eve
year   1953  1955  2011
month    12    11    10
day       3    24    10
```

```
[10]: # Fem servir una llista de llistes per a introduir les dades i dues llistes
      ↳addicionals
      # per a indicar les etiquetes de files i les columnes.
a = [[1953, 12, 3], [1955, 11, 24], [2011, 10, 10]]
print(pd.DataFrame(a, columns=["year", "month", "day"], index = ["alice",
      ↳"bob", "eve"]))
```

```
      year  month  day
alice  1953     12    3
bob    1955     11   24
eve    2011     10   10
```

Cadascuna de les columnes d'un *dataframe* pot tenir tipus de dades diferents, donant lloc a *dataframes* heterogenis:

```
[11]: a = [[1953, "computer science", 3.5], [1955, "archeology", 3.8], [2011,
    ↪ "biology", 2.8]]
print(pd.DataFrame(a, columns=["year", "major", "average grade"], index =
    ↪ ["alice", "bob", "eve"]))
```

	year	major	average grade
alice	1953	computer science	3.5
bob	1955	archeology	3.8
eve	2011	biology	2.8

En versions anteriors de pandas, disposàvem del panell com a tercera estructura de dades. Un **panell** és una estructura de dades **tridimensional** que pot contenir **etiquetes** als eixos i pot ser **heterogènia**.

Actualment, l'ús dels panells està discontinuat, i fem servir sèries i *dataframes* amb índexs multinivell per tal de representar estructures de dades de més de dues dimensions.

Vegem un exemple senzill d'ús d'índexs multinivell per a representar una imatge.

```
[12]: img = [[[0, 0, 0], [0, 15, 0], [0, 0, 15], [15, 0, 0], [180, 180, 180]],
    ↪ [[200, 200, 200], [125, 1, 125], [100, 100, 2], [1, 152, 125], [15, 25,
    ↪ 20]]]

# Fem servir números per indexar les files i les columnes, i les lletres
# "R", "G", "B" per indicar el contingut d'aquest color en cada píxel
index = pd.MultiIndex.from_product(
    [range(len(img)), range(len(img[0])), ["R", "G", "B"]],
    names=['row', 'column', 'color'])

# Creem la sèrie amb l'índex multidimensional especificant les dades
# i els índex
s = pd.Series([x for row in img for col in row for x in col],
    index=index)

s
```

```
[12]: row  column  color
0      0      R      0
      0      G      0
      0      B      0
      1      R      0
      1      G     15
      1      B      0
      2      R      0
      2      G      0
      2      B     15
      3      R     15
```

```

      G      0
      B      0
4     R     180
      G     180
      B     180
1     0     R     200
      G     200
      B     200
      1     R     125
      G      1
      B     125
2     R     100
      G     100
      B      2
3     R      1
      G     152
      B     125
4     R      15
      G      25
      B      20
dtype: int64

```

Visualitzem gràficament la imatge per entendre millor la representació escollida. La imatge té dues files i cinc columnes i per a cada píxel utilitzem tres valors numèrics per representar-ne el color.

```

[13]: %pylab inline
      from pylab import imshow
      imshow(array(img, dtype=uint16), interpolation='nearest')

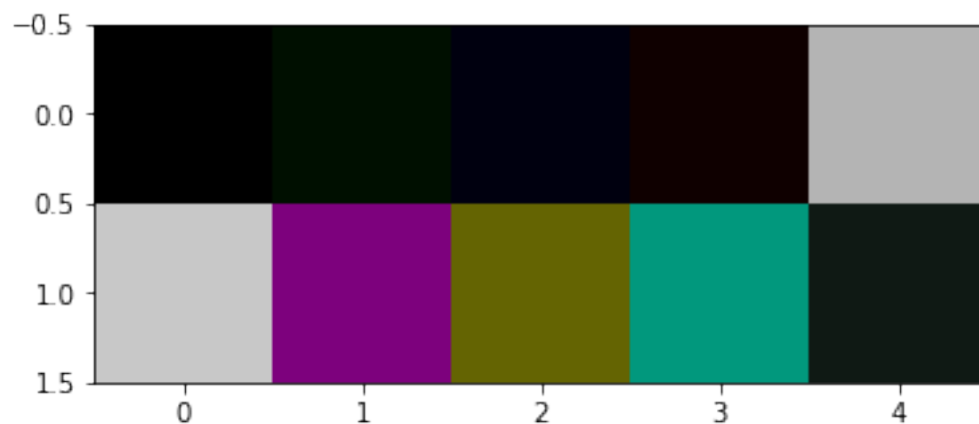
```

Populating the interactive namespace from numpy and matplotlib

```

[13]: <matplotlib.image.AxesImage at 0x7fb3af1beef0>

```



3 Operacions bàsiques sobre un *dataframe*

El *dataframe* és l'estructura més usada a pandas. Vegem algunes de les operacions que podem realitzar amb aquest.

1.2 3.1 Lectura de dades d'un fitxer

Pandas ens permet carregar les dades d'un fitxer CSV directament a un *dataframe* a través de la funció `read_csv`. Aquesta funció és molt versàtil i disposa de multitud de paràmetres per configurar amb tot detall com dur a terme la importació. En molts casos, la configuració per defecte ja ens oferirà els resultats desitjats.

Ara carregarem les dades del fitxer `marvel-wikia-data.csv`, que conté dades sobre personatges de còmic de Marvel. El conjunt de dades va ser creat pel web [FiveThirtyEight](#), que escriu articles basats en dades sobre esports i notícies, i que posa a disposició pública els [conjunts de dades](#) que recull per als seus articles.

```
[14]: # Carreguem les dades del fitxer "marvel-wikia-data.csv" a un 'dataframe'.
data = pd.read_csv("data/marvel-wikia-data.csv")
print(type(data))
```

```
<class 'pandas.core.frame.DataFrame'>
```

3.2 Exploració del *dataframe*

Vegem algunes funcions que ens permeten explorar el *dataframe* que acabem de carregar.

```
[15]: # Mostrar les 3 primeres files.
data.head(n=3)
```

```
[15]:  page_id                                name \
0      1678                Spider-Man (Peter Parker)
1      7139            Captain America (Steven Rogers)
2     64786  Wolverine (James \"Logan\" Howlett)

                                urlslug                ID \
0              \Spider-Man_(Peter_Parker)  Secret Identity
1              \Captain_America_(Steven_Rogers)  Public Identity
2  \Wolverine_(James_%22Logan%22_Howlett)  Public Identity

                                ALIGN                EYE                HAIR                SEX  GSM \
0      Good Characters  Hazel Eyes  Brown Hair  Male Characters  NaN
1      Good Characters  Blue Eyes  White Hair  Male Characters  NaN
2  Neutral Characters  Blue Eyes  Black Hair  Male Characters  NaN

                                ALIVE  APPEARANCES  FIRST APPEARANCE    Year
0  Living Characters             4043.0             Aug-62  1962.0
1  Living Characters             3360.0             Mar-41  1941.0
2  Living Characters             3061.0             Oct-74  1974.0
```

```
[16]: # Mostrar les etiquetes.  
data.index
```

```
[16]: RangeIndex(start=0, stop=16376, step=1)
```

```
[17]: # Mostra estadístiques bàsiques de les columnes numèriques del 'dataframe'.  
data.describe()
```

```
[17]:
```

	page_id	APPEARANCES	Year
count	16376.000000	15280.000000	15561.000000
mean	300232.082377	17.033377	1984.951803
std	253460.403399	96.372959	19.663571
min	1025.000000	1.000000	1939.000000
25%	28309.500000	1.000000	1974.000000
50%	282578.000000	3.000000	1990.000000
75%	509077.000000	8.000000	2000.000000
max	755278.000000	4043.000000	2013.000000

3.3 Indexació i selecció de dades

Podem utilitzar les expressions habituals de Python (i NumPy) per seleccionar dades de *dataframes* o bé fer servir els operadors propis de pandas. Aquests últims estan optimitzats, per la qual cosa el seu ús és recomanat per treballar amb conjunts de dades grans o en situacions en què l'eficiència sigui crítica.

```
[18]: # Seleccionem els noms dels deu primers personatges, és a dir, mostrem la  
      ↳ columna "name" de les deu primeres files  
      # fent servir expressions Python.  
data["name"][0:10]
```

```
[18]: 0          Spider-Man (Peter Parker)  
      1      Captain America (Steven Rogers)  
      2      Wolverine (James \"Logan\" Howlett)  
      3      Iron Man (Anthony \"Tony\" Stark)  
      4              Thor (Thor Odinson)  
      5      Benjamin Grimm (Earth-616)  
      6      Reed Richards (Earth-616)  
      7      Hulk (Robert Bruce Banner)  
      8      Scott Summers (Earth-616)  
      9      Jonathan Storm (Earth-616)  
      Name: name, dtype: object
```

```
[19]: # Seleccionem el nom, l'estat de la seva identitat i el color de cabell dels  
      ↳ superherois 3 i 8  
      # amb l'operador d'accés de pandas .loc  
data.loc[[3,8], ["name", "ID", "HAIR"]]
```



```
[19]:
```

	name	ID	HAIR
3	Iron Man (Anthony \"Tony\" Stark)	Public Identity	Black Hair
8	Scott Summers (Earth-616)	Public Identity	Brown Hair

```
[20]: # Seleccionem files segons el gènere del superheroï utilitzant operadors
      ↪ binaris i expressions Python.
male = data[data.SEX == "Male Characters"]
female = data[data.SEX == "Female Characters"]
print(len(male))
print(len(female))
```

```
11638
3837
```

```
[21]: # Combinem operadors binaris per seleccionar els superheroï amb identitat
      ↪ secreta que han aparegut més
      # de dues mil vegades amb expressions Python.
secret_and_popular1 = data[(data.APPEARANCES > 1000) & (data.ID == "Secret
      ↪ Identity")]
print(len(secret_and_popular1))
print(secret_and_popular1["name"])
```

```
5
0      Spider-Man (Peter Parker)
19     Robert Drake (Earth-616)
23     Piotr Rasputin (Earth-616)
29     Kurt Wagner (Earth-616)
30     Vision (Earth-616)
Name: name, dtype: object
```

```
[22]: # Utilitzem el mètode 'where' de pandas per obtenir la mateixa informació:
secret_and_popular2 = data.where ((data.APPEARANCES> 1000) & (data.ID ==
      ↪ "Secret Identity"))
# Fixeu-vos que en aquest cas el resultat té la mateixa mida que el 'dataframe
      ↪ original': els valors no seleccionats
# mostren NaN.
print(len(secret_and_popular2))
print(secret_and_popular2["name"][0:10])
```

```
16376
0      Spider-Man (Peter Parker)
1                                NaN
2                                NaN
3                                NaN
4                                NaN
5                                NaN
6                                NaN
```

```

7           NaN
8           NaN
9           NaN
Name: name, dtype: object

```

```

[23]: # Podem eliminar les files que tinguin tots els valors NaN, de manera que
      ↳ obtindrem el mateix resultat que fent servir
      # operadors binaris.
      print(secret_and_popular2.dropna(how="all")["name"])

```

```

0      Spider-Man (Peter Parker)
19     Robert Drake (Earth-616)
23     Piotr Rasputin (Earth-616)
29      Kurt Wagner (Earth-616)
30      Vision (Earth-616)
Name: name, dtype: object

```

3.4 Agregació de dades

Pandas també permet crear grups de dades a partir dels valors d'una o més columnes i després operar sobre els grups creats. Vegem alguns exemples.

```

[24]: # Agrupem el 'dataframe' en funció de l'alineació del superheroï.
      grouped = data.groupby("ALIGN")

      # Visualitzem el nom i el nombre de files de cada grup.
      for name, group in grouped:
          print(name, len(group))

```

```

Bad Characters 6720
Good Characters 4636
Neutral Characters 2208

```

```

[25]: # Agrupem el 'dataframe' en funció de l'alineació del superheroï i de
      ↳ l'ocultació de la seva identitat.
      grouped = data.groupby(["ALIGN", "ID"])

      # Visualitzem el nom i el nombre de files de cada grup.
      for name, group in grouped:
          print(name, len(group))

```

```

('Bad Characters', 'Known to Authorities Identity') 3
('Bad Characters', 'No Dual Identity') 474
('Bad Characters', 'Public Identity') 1452
('Bad Characters', 'Secret Identity') 3223
('Good Characters', 'Known to Authorities Identity') 10
('Good Characters', 'No Dual Identity') 647
('Good Characters', 'Public Identity') 1628
('Good Characters', 'Secret Identity') 1613

```

```
('Neutral Characters', 'Known to Authorities Identity') 2
('Neutral Characters', 'No Dual Identity') 390
('Neutral Characters', 'Public Identity') 706
('Neutral Characters', 'Secret Identity') 818
```

```
[26]: # A partir de les dades agrupades, apliquem la funció d'agregació 'np.mean' (que
      ↳ calcula la mitjana).
      grouped.aggregate(np.mean)
```

```
[26]:
```

		page_id	APPEARANCES \
ALIGN	ID		
Bad Characters	Known to Authorities Identity	20762.666667	11.666667
	No Dual Identity	420196.056962	10.434211
	Public Identity	364450.002066	10.846815
	Secret Identity	272270.281415	8.895082
Good Characters	Known to Authorities Identity	67293.700000	72.000000
	No Dual Identity	351762.737249	41.811570
	Public Identity	328621.391892	42.761401
	Secret Identity	209598.569746	37.098220
Neutral Characters	Known to Authorities Identity	25164.000000	180.500000
	No Dual Identity	448877.128205	24.430481
	Public Identity	405297.841360	22.266165
	Secret Identity	255694.179707	19.737113

		Year
ALIGN	ID	
Bad Characters	Known to Authorities Identity	1991.666667
	No Dual Identity	1973.530806
	Public Identity	1973.381458
	Secret Identity	1984.200065
Good Characters	Known to Authorities Identity	1994.100000
	No Dual Identity	1981.512195
	Public Identity	1982.214422
	Secret Identity	1990.303430
Neutral Characters	Known to Authorities Identity	1988.500000
	No Dual Identity	1983.160237
	Public Identity	1987.366322
	Secret Identity	1992.339818

```
[27]: # Recuperem la informació d'un únic grup d'interès.
      grouped.get_group(("Neutral Characters", "Known to Authorities Identity"))
```

```
[27]:
```

	page_id	name	urlslug \
106	18186	Venom (Symbiote) (Earth-616)	\Venom_(Symbiote)_(Earth-616)
2668	32142	Obax Majid (Earth-616)	\Obax_Majid_(Earth-616)

```
ID ALIGN EYE \
```

106	Known to Authorities	Identity	Neutral Characters	Variable Eyes
2668	Known to Authorities	Identity	Neutral Characters	Brown Eyes

	HAIR	SEX	GSM	ALIVE	APPEARANCES	\
106	No Hair	Agender Characters	NaN	Living Characters	348.0	
2668	Black Hair	Female Characters	NaN	Living Characters	13.0	

	FIRST APPEARANCE	Year
106	May-84	1984.0
2668	Sep-93	1993.0

3.5 Escriptura de dades a un fitxer

D'una manera anàloga a com hem carregat les dades d'un fitxer a un *dataframe*, podem escriure les dades d'un *dataframe* a un fitxer CSV.

```
[28]: # Creem un 'dataframe' amb els noms dels superherois.
new_dataset = data[["name"]]
# Guardem el nou 'dataframe' a un fitxer, forçant la codificació a 'utf-8'.
new_dataset.to_csv("marvel-wikia-data-reduced.csv", encoding='utf-8')
```

4 Exercicis i preguntes teòriques

La part avaluable d'aquesta unitat consisteix en el lliurament d'un fitxer IPython Notebook amb extensió IPYNB que contindrà els diferents exercicis i les preguntes teòriques que s'han de contestar. Trobareu el fitxer (`prog_datasci_4_scilib_entrega.ipynb`) amb les activitats a la mateixa carpeta que aquest notebook que esteu llegint. **Hi ha un sol fitxer d'activitats per a tota la unitat, que cobreix totes les llibreries que s'hi treballen.**

4.1 Instruccions importants

És molt important que a l'hora de lliurar el fitxer Notebook amb les vostres activitats us assegureu que:

1. Les vostres solucions siguin originals. Esperem no detectar-hi còpia directa entre estudiants.
2. Tot el codi estigui correctament documentat. El codi sense documentar equivaldrà a un 0.
3. El fitxer comprimit que lliureu és correcte (conté les activitats de la PAC que heu de lliurar).

Per fer el lliurament, heu d'anar a la carpeta del drive **Colab Notebooks**, clicant botó dret a la PAC en qüestió i fent **Download**. D'aquesta manera, us baixareu la carpeta de la PAC comprimida en zip. Aquest és l'arxiu que heu de pujar al campus virtual de l'assignatura.

2 Autors

- Autora original **Cristina Pérez Solà**, 2017
- Actualitzat per **Cristina Pérez Solà**, 2019

<div style="width:0%;"> </div>

</div>